

Invited Paper: MLCAD 2025 Contest on ReSynthAI: Physical-aware Logic Resynthesis using AI

Vikram Gopalakrishnan¹, Atmadip Dey¹, Rongjian Liang², Yanqing Zhang², Haoxing Ren², and Vidya A. Chhabria¹
¹Arizona State University; ²NVIDIA Research

Abstract—As Moore’s Law slows, enhancements in power, performance, and area (PPA) increasingly rely on innovations in EDA rather than process scaling. Logic optimization is critical in the RTL-to-GDSII flow, yet traditional approaches remain heuristic-driven and time-consuming. Recent advances in machine learning (ML) and GPU computing present new opportunities to transform logic optimization. By leveraging artificial intelligence (AI) — encompassing both advanced ML techniques and GPU-accelerated computation — a wide range of physically-aware resynthesis strategies becomes possible. These include predicting downstream impacts of early-stage design decisions, using generative models to propose logic transformations directly, and accelerating placement and routing to obtain physical feedback rapidly. This paper introduces the MLCAD 2025 Contest, ReSynthAI, which incorporates post-global-route physical awareness into logic resynthesis after floorplanning. The contest provides a complete infrastructure, including benchmarks in standard EDA and ML-friendly formats, and examples of “ML inside” EDA tools using Python APIs. It leverages OpenROAD and CircuitOps, enabling participants to build on accessible, open-source tools and data formats. With 27 registered teams, the contest demonstrates strong momentum in ML-driven timing optimization and its growing community impact.

I. INTRODUCTION

With scaling and the slowdown in Moore’s Law, electronic design automation (EDA) tools must work increasingly harder to achieve power, performance, and area (PPA) specifications. A critical step for PPA improvement is logic optimization, which, today, remains heuristic-based and may not result in optimal solutions. It can consist of multiple transformations to the logic gate-level netlist, including logic restructuring, logic gate sizing, Vt swapping, buffer insertion/deletion, pin swapping, and gate cloning. These transformations are demonstrated in Fig. 1. Identifying which transformation or a combination of transformations during each iteration of the logic optimization algorithm is a very challenging problem. The problem is challenging due to (i) the non-convexity of the delay models, (ii) the discrete space of logic gate choices and transformations, (iii) the number of near-critical paths, (iv) the tradeoff between power and timing, and (v) the resource availability constraints which prevent the exploration of the large design space.

Logic or netlist optimization has been an active area of research for over three decades. Academic efforts have predominantly focused on isolated transformations, such as logic gate sizing [1]–[5] and buffer insertion [6]. While a

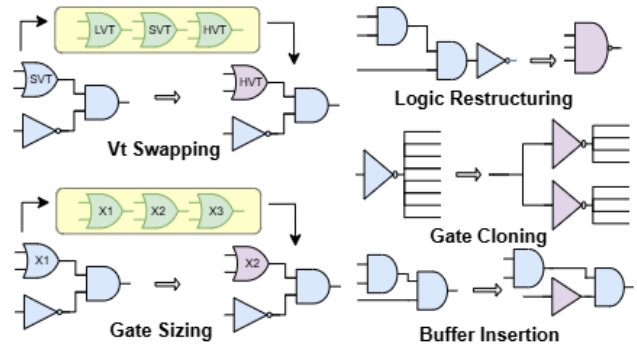


Fig. 1. Possible transformations on a gate-level netlist (not exhaustive).

few studies have jointly considered two transformations at a time, gate sizing and buffer insertion [7], [8], gate sizing and Vt swapping [9], approaches that simultaneously optimize across all available timing transformations remain scarce. This narrow scope significantly limits exploration of the whole design space. Moreover, most existing techniques lack physical awareness, often overlooking critical downstream effects such as parasitics and congestion. Although there has been some work on physical resynthesis [10]–[12], these methods typically rely on heuristics that trade off runtime for solution quality.

Recent advances have introduced machine learning (ML) or artificial intelligence (AI) techniques into netlist optimization. For instance, reinforcement learning (RL) has been applied to gate sizing [13], [14], but suffers from scalability limitations. Other ML-based methods have been explored in gate sizing [15], [16] and buffer insertion [17]. However, these techniques also share the same limitations: they treat transformations in isolation and lack physical awareness.

There is thus a pressing need to develop physically-aware logic optimization algorithms capable of jointly considering a wide range of transformations, without being restricted to a single category. Contests have proven to be effective drivers of research innovation in both EDA and broader AI communities [18], [19]. Previous contests on logic or netlist optimization have focused mainly on isolated transformations, such as logic gate sizing [20]–[22], and often rely on unrealis-

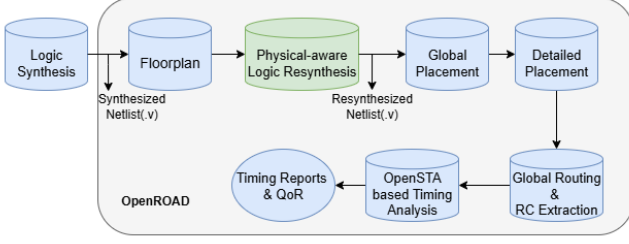


Fig. 2. ReSynthAI focuses on improving post-global-route PPA.

tic or partially constructed benchmarks that do not incorporate physical effects.

Similar to the ICCAD 2024 contest [22], our contest, ReSynthAI [23], [24], focuses on leveraging AI — encompassing both advanced ML techniques and GPU accelerated computation — techniques to improve power while satisfying timing constraints. However, ReSynthAI goes beyond previous contests by addressing the critical challenge of physical awareness. Specifically, our contest targets optimization post the floorplan stage, while evaluation incorporates placement and global routing, including parasitic extraction at the post-global-route stage, as illustrated in Fig. 2. A key differentiator of ReSynthAI is its open design space: participants are allowed to employ any transformation—such as logic gate sizing, buffer insertion, logic restructuring, or threshold voltage (Vt) swaps (Fig. 1)—as long as the resynthesized netlist remains logically equivalent to the original. In contrast, previous contests have typically constrained participants to a single type of transformation and relied on pre-synthesis or netlist-only evaluations. To further broaden accessibility, ReSynthAI builds on open-source infrastructure by providing Python APIs that interface with established EDA tools, along with AI-friendly data representations. This reduces barriers to entry and encourages participation from a diverse research community, including those outside traditional EDA domains. The ReSynthAI contest serves four primary purposes:

- 1) Lowers the barrier to entry for non-EDA experts by converting traditional EDA problems into ML-solvable tasks through the use of ML-friendly data formats.
- 2) Leverages existing infrastructure to enable true “ML inside” EDA tool integration, utilizing OpenROAD Python APIs and CircuitOps as demonstrated in [22], [25].
- 3) Explores research algorithms that leverage ML or GPU acceleration to advance timing optimization while simultaneously considering logic restructuring, gate sizing, Vt swaps, buffer insertion/deletion, and pin swapping.
- 4) Leverages fast and scalable logic equivalence checking using a GPU-accelerated gate-level simulator [26]. Formal verification is conducted only for contest winners to guarantee logical equivalence with timely feedback.
- 5) Releases benchmarks in an academic FinFET technology node (ASAP7 [27]), along with evaluation scripts [28].
- 6) Incentivizes the development of physically-aware logic resynthesis that account for downstream impacts of

placement and global routing, thus aligning research more closely with real-world physical design challenges.

The contest releases a set of logic gate-level (five hidden and seven visible) benchmark netlists for timing in ASAP7 technology node. The benchmark netlists are provided in (i) standard EDA file format, which includes post-floorplan DEF, gate-level netlists, constraints (.sdc file), and (ii) CircuitOps [29] intermediate representation table (IR Table) format. The evaluation scripts, which use OpenROAD [30] to perform placement, global routing, RC extraction, and timing analysis, are also provided. Further, example scripts to interact with OpenROAD via Python APIs are also provided for various transformations. In this contest, participants must develop a scalable logic resynthesis engine to minimize leakage power of the design, while meeting timing, maximum load capacitance, and slew constraints. The contest develops a cost function as described in Section IV, which is the scoring criterion for the contest and is evaluated after global route.

II. CONTEST PROBLEM DESCRIPTION

A. Problem Formulation

The objective for contest participants is to develop a logic resynthesizer that solves the constrained optimization problem: Objective:

$$\min \sum_{i \in I} P_{\text{leak}}(c_i)$$

subject to:

$$\begin{aligned} \text{slack}(p_i) &\geq 0, \quad \forall i \in I, \\ s(p_i) &\leq s_{\text{max}}(p_i), \quad \forall i \in I, \\ L(p_i) &\leq L_{\text{max}}(p_i), \quad \forall i \in I, \\ O_{\text{tot}} &= 0. \end{aligned}$$

where:

- I is the set of instances in the design.
- C_i is the set of library cells available for instance i .
- $c_i \in C_i$ is the library cell assigned to instance i .
- $P_{\text{leak}}(c_i)$ is the leakage power of instance i when implemented with cell c_i .
- $\text{slack}(p_i)$ is the timing slack at pin p of instance i .
- $s(p_i)$ is signal transition (slew) at pin p of instance i .
- $s_{\text{max}}(p_i)$ is the maximum allowed slew at pin p_i (from the library).
- $L(p_i)$ is the load capacitance at pin p of instance i .
- $L_{\text{max}}(p_i)$ is the maximum allowed load capacitance at pin p_i (from the library).
- O_{tot} is the total number of global routing overflows.

All the above metrics are measured after global routing using available global-route-based parasitic estimates. The goal is to minimize the total leakage power while ensuring that slack is non-negative, slew and load meet the maximum allowed transition time and max load capacitance for each pin without having global route overflows. The optimization is performed after floorplanning stage. In addition to the above constraints, the resynthesized netlist must be logically equivalent to the original netlist post logic synthesis.

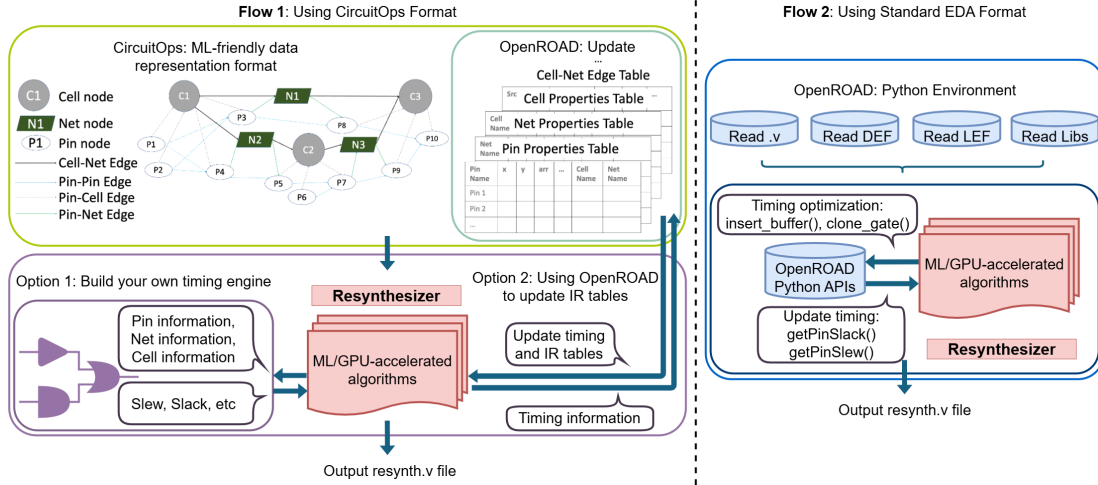


Fig. 3. Proposed flows for developing the resynthesizer using the two input formats

B. Inputs Description

The contest provides inputs in two formats. The first is standard EDA files, and the second is CircuitOps' IR table format. Depending on the choice of input that participants choose to use we propose two flows as shown in Fig. 3. The first flow uses CircuitOps and the second uses the standard EDA file format. Below, we describe the two input formats and the suggested flows:

(i) Standard EDA file format: In this contest, EDA files, including post-floorplan DEF (.def), gate-level netlist (.v), constraints (.sdc), and the technology files from ASAP7 as shown in Fig. 3 (right). If participants choose to work with these files, they can develop their parsers to read them. If participants develop their resynthesis engine in C/C++, they can leverage OpenROAD and OpenSTA header files to parse liberty and netlist. If participants decide to develop solutions in Python, they can use OpenROAD Python APIs to query the necessary information from the files [25] (Fig. 3(right)), or they can reuse existing Python packages such as liberty-parser [31] and verilog-parser [32]. Participants can also work with their parsers if they cannot extract the information they need from existing APIs. In this contest [24], example API scripts for querying information from OpenROAD are provided if participants choose to use OpenROAD as the parser and work within the OpenROAD Python interpreter. The corresponding example transformation scripts using OpenROAD for all transformations shown in Fig. 1 are also provided using OpenROAD Python APIs [24].

(ii) CircuitOps IR table format: The CircuitOps data representation format [33] provides an ML-friendly format consisting of graphs and annotated features as shown in Fig. 3(left). The graph is a labeled property graph (LPG) that is backed by IR tables [33]. The graph utilizes a node for a net, pin, and cell of a circuit. It has five types of edges to represent connections between pin-to-pin nodes (pin-pin), pin-to-cell nodes (pin-cell), pin-to-net nodes (pin-net), cell-to-net nodes (cell-net), and cell-to-cell nodes (cell-cell). Each node and edge has associated IR tables with properties associated with the node

and edge. The IR tables of all benchmarks are provided to the contestants, which can easily be parsed into their resynthesizer using standard CSV readers in C++/Python. These files are in a CSV format [33] and can be read as Pandas dataframes. If participants choose to work with CircuitOps data representation format as input, they do not need to understand complex EDA formats. However, this would require iterative interaction with the OpenROAD timer to update the IR tables or participants to develop their own timer. To support working with this format, we have provided APIs that interact with OpenROAD to generate intermediate files or updated IR tables at regular intervals during the participant's solution iterations. The choice of the input format is left to the participants. The EDA file-based format will provide you with maximum information to develop the resynthesizer and the ability to extract custom features for ML models. The CircuitOps format will be limited to the features and properties available in CircuitOps, but it will allow participants to skip the tedious parsing tasks and focus on the ML/GPU acceleration aspects. The contest has released translators from the standard EDA file format to CircuitOps format (the translators leverage OpenROAD APIs and database queries).

C. Intermediate Files Description

If contestants are using Flow 1 (CircuitOps) as shown in Fig. 3(left) and choose not to build their own timing engine, we provide an IR table generator. This tool takes the updated netlist as input, performs timing analysis using default floorplan.def, and generates new IR tables in .csv format.

D. Output Description

The output file is an updated gate-level Verilog netlist (.v).

E. Support Scripts

The contest provided two types of support scripts for both Flow 1 and Flow 2 shown in Fig. 3.

(i) Flow 1: Using EDA files as input, these scripts read the design files, along with modified netlist, and provide updated CircuitOps relational tables with either file I/Os (input: EDA

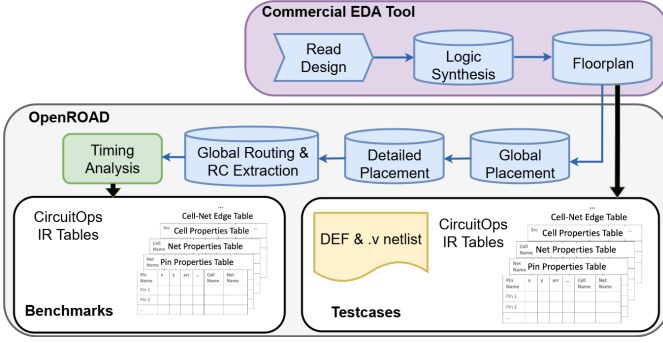


Fig. 4. Flow for generating the contest benchmarks.

file formats and output: intermediate file tables CSVs) or directly with Pandas dataframes as outputs.

(ii) Flow 2: Use standard EDA tool files as input, the provided script gives examples to operate with OpenROAD Python APIs to perform transformations shown in Fig. 1 and obtain updated timing and power numbers. These scripts work directly as function calls and return standard Python data types.

III. BENCHMARKS

In this contest, benchmarks are released in the ASAP7 technology node [27]. The benchmarks vary in size from 10K to 150K instances. There are seven visible benchmarks and five hidden benchmarks, which are used to evaluate participants' solutions. The hidden benchmarks will remain undisclosed during the contest to ensure fair evaluation. They will be released publicly after the contest concludes. The benchmarks can be found at [24]. All benchmarks are synthesized with rising-edge-triggered flip-flops only.

The benchmarks were taken from the TILOS MacroPlacement repository [34] and the IWLS 2005 benchmarks [35]. The RTL for these designs is synthesized to gate-level netlists and then floorplanned using commercial EDA tools. Next, the design is taken down the flow until global routing as shown in Fig. 4 using OpenROAD [30]. The original post-floorplan netlists, along with the corresponding floorplan DEF and constraints files, are released, along with CircuitOps IR tables. The timing values post global route are provided as benchmarks, and IR tables for global route designs are also included. The timing numbers are derived from OpenROAD's timing engine, OpenSTA, using post-global route parasitics from OpenROAD.

It is important to note that wire load models are not used in these benchmarks, leading to an optimistic timing estimation during the floorplan stage. While the timing values at the floorplan stage are optimistic, the post-global route timing is more pessimistic due to parasitics estimated by OpenROAD. Since the post-global route metrics are considered the final evaluation metrics, this introduces an additional challenge for participants to develop accurate alternatives to wire load models, thereby emphasizing the significant impact of physical awareness and parasitics on logic resynthesis.

Each benchmark in both the public and hidden test cases varies in terms of gate counts, as well as initial power and

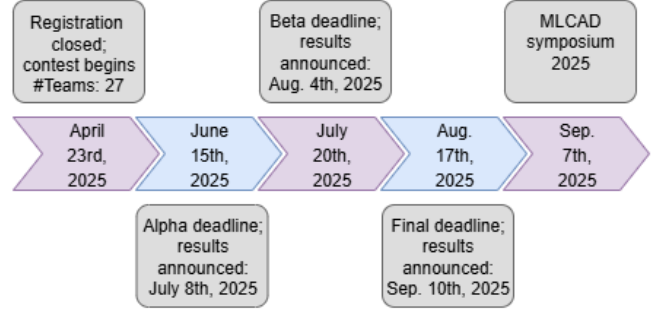


Fig. 5. MLCAD25 ReSynthAI contest timeline.

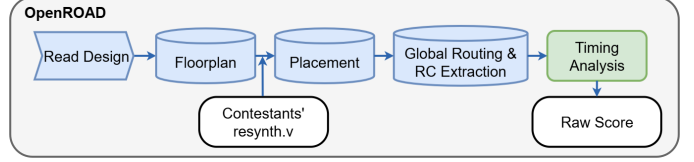


Fig. 6. Contest evaluation flow.

timing parameters. The power and timing statistics for the public and hidden benchmarks after the global route stage are shown in Table I. The timing and leakage power values are reported by OpenSTA. Other columns in the table list the total slew violation difference and total load violation difference of a baseline optimization, which are defined as:

$$V_{\text{slew}}^{(0)} = \sum_i [s(p_i) - s_{\max}(p_i)], \quad \forall p_i \text{ when } s(p_i) > s_{\max}(p_i)$$

$$V_{\text{load}}^{(0)} = \sum_i [L(p_i) - L_{\max}(p_i)], \quad \forall p_i \text{ when } L(p_i) > L_{\max}(p_i)$$

The values of $s_{\max}(p_i)$ and $L_{\max}(p_i)$ are specific to the pins of each library cell. These values are included as part of the CircuitOps tables and can also be queried using OpenROAD.

IV. CONTEST EVALUATION

Assumptions: The contest focus is on optimizing setup time only. Sequential gate optimization is not permitted. E.g., sizing of macros or SRAM cells is not allowed. All sequential gates in the benchmark are rising-edge triggered. The benchmarks assume an ideal clock with no clock buffers in the circuit (zero skew). Our evaluation flow is shown in Fig. 6. The evaluation script first ensures logical equivalence, then reads the resynthesized netlist along with the default floorplan.def, then performs placement, global routing, updates the global route parasitics, and estimates the score described below.

Logical equivalence: Each generated resynthesized netlist is compared against the original post-synthesis netlist to ensure logical equivalence. Traditional formal verification techniques are not scalable for timely evaluation across all participants and designs. Therefore, for the ReSynthAI contest, we employ a subset of GLOAM from NVLabs [26], [36] to verify logical equivalence between the synthesized and resynthesized netlists. GLOAM is a GPU-accelerated logic cone simulator

TABLE I
BENCHMARK BASELINE STATISTICS POST GLOBAL ROUTE.

Design	#instances	Clock period (ns)	WNS (ns)	TNS (ns)	$V_{\text{slew}}^{(0)}$ (ns)	$V_{\text{load}}^{(0)}$ (fF)	$P_{\text{leak}}^{(0)}$ (μW)	$O_{\text{tot}}^{(0)}$
ac97 top	8,029	0.120	-0.068	-17.921	0.506	0.000	82.793	0
aes_cipher top	10,965	0.320	-0.160	-18.846	9.769	6.589	29.763	0
NV_NVDLA partition_m	17,843	0.400	-0.209	-11.906	54.497	344.927	161.323	0
fpu	19,725	3.680	-1.208	-59.335	23.786	4.546	19.254	0
NV_NVDLA partition_p	58,216	1.000	-0.492	-747.111	1,386.116	7,692.260	579.834	0
NV_NVDLA partition_c	155,686	3.000	-0.722	-18.825	20,854.780	66,908.071	18,099.664	0
NV_NVDLA partition_p mod	58,026	1.200	-0.431	-391.007	1,199.430	7,483.045	579.445	0
Hidden1	12,130	0.600	-0.423	-0.423	1.788	0.000	52.897	0
Hidden2	32,795	0.800	-0.338	-139.020	5,410.290	8,087.704	347.034	0
Hidden3	112,896	2.500	-0.985	-3,532.406	17,472.955	18,008.632	17,539.106	0
Hidden4	150,545	3.000	-1.907	-13,747.954	1,909.798	1,446.475	3129.803	0
Hidden5	15,236	0.500	-0.196	-7.304	41.731	298.254	129.580	0

that extracts the combinational directed acyclic graph (DAG) from the netlist and performs partitioning to create independent logic cone graphs. These smaller partitions are mapped to individual GPU thread blocks, enabling simulation with randomly generated stimuli to produce output waveforms. This simulation is performed for both the synthesized netlist and the contestant's resynthesized netlist, and the resulting output waveforms are compared to confirm logical equivalence. While simulation-based LEC is fast, it does not ensure logical equivalence. Therefore, we leverage simulation-based LEC for all participants and verify the winner's solution using formal verification tools to ensure fairness and timely evaluations.

Score: Each resynthesizer is evaluated based on the quality of the generated solution and the time taken to generate the solution. The following score is used to rank solutions.

$$S = (1 + F_t) \times \left[\frac{P_{\text{leak}}}{P_{\text{leak}}^{(0)}} + 10 \cdot \frac{|TNS|}{|TNS^{(0)}|} + 20 \cdot \frac{V_{\text{slew}}}{V_{\text{slew}}^{(0)}} + 20 \cdot \frac{V_{\text{load}}}{V_{\text{load}}^{(0)}} + 10 \cdot \delta \cdot (O_{\text{tot}} - O_{\text{tot}}^{(0)}) \right], \quad (1)$$

where

$$V_{\text{slew}} = \sum_i \beta_i [s(p_i) - s_{\text{max}}(p_i)], \quad (2)$$

$$V_{\text{load}} = \sum_i \gamma_i [L(p_i) - L_{\text{max}}(p_i)], \quad (3)$$

$$\beta_i = \begin{cases} 0, & s(p_i) \leq s_{\text{max}}(p_i), \\ 1, & \text{otherwise}, \end{cases} \quad (4)$$

$$\gamma_i = \begin{cases} 0, & L(p_i) \leq L_{\text{max}}(p_i), \\ 1, & \text{otherwise}, \end{cases} \quad (5)$$

$$\delta = \begin{cases} 0, & O_{\text{tot}} - O_{\text{tot}}^{(0)} \leq 0, \\ 1, & \text{otherwise}, \end{cases} \quad (6)$$

$$F_t = \min \left\{ 0.2, \max \left[0.02 \cdot \log_2 \left(\frac{t}{t_{\text{med}}} \right), -0.2 \right] \right\}. \quad (7)$$

All design metrics are normalized by their corresponding

TABLE II
NOTATION USED IN THE SCORE FUNCTION DEFINED IN EQUATIONS (1)

Symbol	Description
S	Final evaluation score.
t	Runtime of the resynthesis tool flow in secs.
t_{med}	Median runtime across all teams in secs.
F_t	Runtime factor penalty.
P_{leak}	Leakage power of the design in μW .
$P_{\text{leak}}^{(0)}$	Baseline leakage power in μW .
TNS	Total negative slack in ns.
$TNS^{(0)}$	Baseline total negative slack in ns.
V_{slew}	Total slew violation magnitude in ns.
$V_{\text{slew}}^{(0)}$	Baseline slew violation magnitude in ns.
β_i	Slew violation indicator for pin p_i .
$s(p_i)$	Measured slew at pin p_i in ns.
$s_{\text{max}}(p_i)$	Allowed maximum slew at pin p_i in ns.
V_{load}	Total load violation magnitude in fF.
$V_{\text{load}}^{(0)}$	Baseline load violation magnitude in fF.
γ_i	Load violation indicator for pin p_i .
$L(p_i)$	Measured load at pin p_i in fF.
$L_{\text{max}}(p_i)$	Allowed maximum load at pin p_i in fF.
O_{tot}	Total routing overflow.
$O_{\text{tot}}^{(0)}$	Baseline total routing overflow.
$O(g)$	Overflow in GCell g .
δ	Routing overflow penalty indicator.

baseline values obtained from the original netlist after global routing. All the variables used in the table are defined in Table II. Baseline values for the provided benchmarks are in Table I. For each team, each benchmark is then scored out of 100 where the team that has the least score for a given benchmark receives a 100 points, and all other team scores are graded relative to that, i.e., the total score for a specific benchmark is given by: $100 \times \frac{\text{Least score for the benchmark}}{\text{Score for the benchmark}}$. In the contest, if a participant's solution does not generate the resynth.v file within the 3-hour limit or the proposed solution

does not finish global route within the 3-hour limit, a score of 0 out of hundred is awarded. In the process of calculating the median runtime, we did not take the runtime of failed cases into account. The scores are added across all testcases, and the top three teams with the highest scores will win the contest.

Platform: The submitted resynthesizers are run on a computing platform equipped with four A100 GPUs, each with an 80 GB memory capacity. The utilization of up to 8 CPU threads is supported. We supply a Docker image preconfigured with CUDA and some popular ML libraries to facilitate a standardized development environment. We encourage participating teams to capitalize on the potential of GPU acceleration and explore the integration of ML techniques. However, the usage of the GPU is optional. For evaluation, we limited to 4 GPUs and 8 CPU threads due to computation challenges.

Evaluation Scripts: We release evaluation scripts for participants to test scores on visible benchmarks. The runtime-related components are evaluated on the platform and compared with those of other participants.

V. CONTEST LOGISTICS

The contest commenced on 23rd April 2025 with a total of 27 officially registered teams. The overall contest timeline is shown in Fig. 5. For the *alpha* submission phase, we received seven valid submissions, and in the *beta* submission phase, seven teams submitted again, including some with updated tool flows and optimizations. The leaderboards for all submission phases are published in the contest benchmarks repository, enabling teams to compare their solutions with others.

The contest was structured in multiple phases, beginning with the release of benchmark circuits, scoring methodology, and baseline reference results. Participants were provided with

detailed documentation, example scripts, and the scoring infrastructure to ensure reproducibility of results. All evaluation runs were performed on standardized hardware to ensure fairness, with runtime limits and memory constraints enforced uniformly. For transparency, all contest logistics, rules, deadlines, and updates were maintained on the official contest website [23] and contest GitHub [24], which also served as the central hub for announcements, Q&A, and technical clarifications. The hidden benchmarks used will be released in the contest GitHub after the contest concludes.

VI. CONCLUSION

The contest establishes the infrastructure necessary to advance ML/AI and GPU-accelerated research in logic resynthesis by providing benchmarks in both standard EDA file formats and ML-compatible formats, along with supporting scripts that demonstrate how to integrate "ML inside" an open-source EDA tool. These benchmarks are released using an open-source 7nm FinFET PDK. Participants are tasked with developing a resynthesizer that minimizes circuit leakage power while adhering to constraints on slack, maximum load, maximum slew, and congestion. We aim for this contest to catalyze research in ML/AI for EDA and GPU-accelerated optimization. We hope that the contest will inspire future innovations in physical-aware logic resynthesis research.

VII. ACKNOWLEDGEMENT

We extend our gratitude to MLCAD 2025 organizers for the invitation to organize the contest and for their support. We thank Dr. Zhiang Wang (UCSD) for reviewing our contest statement. We also thank NVIDIA for sponsoring the GPUs awarded to contest winners.

REFERENCES

- [1] J. P. Fishburn and A. E. Dunlop, "TILOS: A Posynomial Programming Approach to Transistor Sizing," in *Proc. ICCAD*, 1985.
- [2] J. Hu, *et al.*, "Sensitivity-Guided Metaheuristics for Accurate Discrete Gate Sizing," in *Proc. ICCAD*, 2012.
- [3] K. Kasamsetty, *et al.*, "A New Class of Convex Functions for Delay Modeling and its Application to the Transistor Sizing Problem," *IEEE T. Comput. Aid. D.*, vol. 19, no. 7, 2000.
- [4] A. Sharma, *et al.*, "Fast Lagrangian Relaxation-Based Multithreaded Gate Sizing Using Simple Timing Calibrations," *IEEE T. Comput. Aid. D.*, vol. 39, no. 7, 2020.
- [5] C.-P. Chen, *et al.*, "Fast and Exact Simultaneous Gate and Wire Sizing by Lagrangian Relaxation," *IEEE T. Comput. Aid. D.*, vol. 18, no. 7, 1999.
- [6] L. Zhang, *et al.*, "Sampling-based Buffer Insertion for Post-Silicon Yield Improvement under Process Variability," in *Proc. DATE*, pp. 1205–1208, 2017.
- [7] H. Wu, *et al.*, "AiTO: Simultaneous gate sizing and buffer insertion for timing optimization with GNNs and RL," *Integration*, vol. 98, p. 102211, 2024.
- [8] K. Lowe and P. Gulak, "A Joint Gate sizing and Buffer Insertion Method for Optimizing Delay and Power in CMOS and BiCMOS Combinational Logic," *IEEE T. Comput. Aid. D.*, vol. 17, no. 5, pp. 419–434, 1998.
- [9] Y. Liu and J. Hu, "A New Algorithm for Simultaneous Gate Sizing and Threshold Voltage Assignment," in *Proc. ISPD*, p. 27–34, 2009.
- [10] T. Kutzschebauch and L. Stok, "Congestion-aware layout-driven logic synthesis," in *Proc. ICCAD*, pp. 216–223, 2001.
- [11] H. Pan, *et al.*, "Physically Aware Synthesis Revisited: Guiding Technology Mapping with Primitive Logic Gate Placement," in *Proc. ICCAD*, 2025.
- [12] S. Chatterjee and R. K. Brayton, "A New Incremental Placement Algorithm and Its Application to Congestion-Aware Divisor Extraction," in *Proc. ICCAD*, pp. 541–548, 2004.
- [13] Y.-C. Lu, *et al.*, "RL-Sizer: VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning," in *Proc. DAC*, 2022.
- [14] W. Jiang, *et al.*, "IR-Aware ECO Timing Optimization Using Reinforcement Learning," in *Proc. MLCAD*, 2024.
- [15] X. Zhou, *et al.*, "Heterogeneous Graph Neural Network-Based Imitation Learning for Gate Sizing Acceleration," in *Proc. ICCAD*, 2022.
- [16] Z. Zhang, *et al.*, "LAG-Sizer: A Novel Gate Sizer Based on Leak Generative Adversarial Network with Feature Fusion," in *Proc. ICCAD*, 2025.
- [17] R. Liang, *et al.*, "BufFormer: A Generative ML Framework for Scalable Buffering," in *Proc. ASP-DAC*, pp. 264–270, 2023.
- [18] G. Rahmaniani, *et al.*, "Invited: Mapping Two Decades of Innovation: Lessons from 25 Years of ISPD Research," in *Proc. ISPD*, p. 102–110, 2025.
- [19] N. Karimpour Darav, *et al.*, "The impact of industry-organized contests on eda education," in *IEEE International Conference on Microelectronics Systems Education (MSE)*, pp. 21–24, 2015.
- [20] M. M. Ozdal, *et al.*, "The ISPD-2012 Discrete Cell Sizing Contest and Benchmark Suite," in *Proc. ISPD*, 2012.
- [21] M. M. Ozdal, *et al.*, "An Improved Benchmark Suite for the ISPD-2013 Discrete Cell Sizing Contest," in *Proc. ISPD*, 2013.
- [22] B.-Y. Wu, *et al.*, "Invited Paper: 2024 ICCAD CAD Contest Problem C: Scalable Logic Gate Sizing using ML Techniques and GPU Acceleration," in *Proc. ICCAD*, 2024.
- [23] "MLCAD25-Contest." <https://github.com/ASU-VDA-Lab/MLCAD25-Contest>, 2025.
- [24] "MLCAD 2025 Contest on ReSynthAI: Physical-aware Logic Resynthesis using AI." <https://github.com/ASU-VDA-Lab/MLCAD25-Contest-Scripts-Benchmarks>, 2025.
- [25] V. A. Chhabria, *et al.*, "Invited Paper: OpenROAD and CircuitOps: Infrastructure for ML EDA Research and Education," in *Proc. VTS*, 2024.
- [26] "GL0AM." https://github.com/NVlabs/GL0AM/tree/mlcad2025_contest_sim, 2024.
- [27] L. T. Clark, *et al.*, "ASAP7: A 7-nm FinFET Predictive Process Design Kit," *Microelectronics Journal*, vol. 53, 2016.
- [28] "ICCAD 2024 Contest Gate Sizing Benchmarks." https://github.com/ASU-VDA-Lab/2024_ICCAD_Contest_Gate_Sizing_Benchmark/, 2024.
- [29] R. Liang, *et al.*, "Invited Paper: CircuitOps: An ML Infrastructure Enabling Generative AI for VLSI Circuit Optimization," in *Proc. ICCAD*, 2023.
- [30] T. Ajayi, *et al.*, "INVITED: Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project," in *Proc. DAC*, 2019.
- [31] "Liberty-parser." <https://pypi.org/project/liberty-parser/>, 2019.
- [32] "Verilog-parser." <https://pypi.org/project/verilog-parser/>, 2021.
- [33] "CircuitOps." <https://github.com/NVlabs/CircuitOps>, 2024.
- [34] "TILOS MacroPlacement." <https://github.com/TILOS-AI-Institute/MacroPlacement>, 2024.
- [35] "IWLS 2005 Benchmarks." <https://iwls.org/iwls2005/benchmarks.html>, 2005.
- [36] Y. Zhang, *et al.*, "GL0AM: GPU Accelerated Gate Level Logic Simulator," in *Proc. ICCAD*, 2024.